

# Adora — CPU Performance & Scaling

1.5D Stokes synthesis benchmark · Ivan Milic · 2026-08-28 · benchmark note

**What was measured.** Column-by-column 1.5D full-Stokes synthesis ([synth3d.py](#)) of a  $100 \times 100$  atmosphere cube: a FALC column tiled across the map with velocity/field ramps, Fe I 630.15/630.25 nm, 101 wavelengths,  $n_z = 82$  depths — i.e.  $10^4$  independent columns, each a polarised DELO solve through the differentiable stack. Hardware: a 256-core CPU node; env `adora-develop` (JAX 0.7.1, `fp64`). The engine is unchanged — only *how the columns are distributed* across cores varies.

## Headline: cost per pixel

A “pixel” is one full column (all wavelengths, all four Stokes parameters). Single-device JAX runs one column in  $\approx 6.5$  ms but only reaches  $\sim 8$  effective cores — most of XLA’s CPU threads *spin-wait*, because this kernel exposes only  $\sim 2$  cores of real intra-instance parallelism. Spreading the  $10^4$  columns over independent, core-pinned worker processes collapses that to **219  $\mu$ s / pixel** — a  $\sim 30\times$  **speed-up** — and genuinely uses the cores.

## Process-parallel scaling ( $100 \times 100$ )

Total core budget fixed at 64; workers pinned to disjoint core sets with `os.sched_setaffinity`, launched via `multiprocessing.spawn` from a JAX-free parent.

| proc<br>$\times$ | core<br>/proc | comp.<br>(s) | run<br>(s) | $\mu$ s/<br>px | eff.<br>cores |
|------------------|---------------|--------------|------------|----------------|---------------|
| 8                | 8             | 11.5         | 10.51      | 1051           | 15.4          |
| 16               | 4             | 7.4          | 6.17       | 617            | 24.5          |
| 32               | 2             | 6.6          | 3.34       | 334            | 41.2          |
| 64               | 1             | 6.4          | 2.19       | 219            | 55.7          |

`run` = wall for one full  $100 \times 100$  Stokes cube (concurrent workers); `comp.` is the one-time JIT compile. All outputs finite and identical in shape (100, 100, 4, 101). **64 single-core processes win:**  $100 \times 100$  in **2.2 s** at **55.7/64 effective cores** (87% efficiency). Fewer cores per worker  $\Rightarrow$  less spin-wait wasted  $\Rightarrow$  near-linear scaling.

## What did *not* work, and why

- **Single XLA device (default).** Auto-multithreads to only  $\sim 8$  effective cores on this kernel and can go no wider; the batch `vmap` over columns is not exposed as core-level parallelism.
- **`pmap` over 64 host CPU “devices”.** Hit the 64-core target, but pathological: **366 s** to compile the fused `fp64` program and **>200 s** per run — i.e. *slower per pixel than 8 cores*. Naive `pmap-of-vmap` is a dead end on XLA-CPU here.
- **Process-level data parallelism (the winner).** Independent single-device workers, one core each, pinned to disjoint cores. No giant fused kernel, 6 s compile, near-linear scaling. The parent stays numpy-only so `spawn’d` workers start with a clean JAX.

## Takeaways for the 3D extension

- Scale by **running many single-core column workers**, not by widening one XLA computation.
- **Projection:** at 219  $\mu$ s/px a real  $512 \times 512$  snapshot ( $2.6 \times 10^5$  columns)  $\approx 57$  s/run + one  $\sim 6$  s compile — vs.  $\sim 28$  min single-core.
- Next: fold a `synth_cube_mp(cube, waves, dz, nworkers)` helper beside [synth3d.py](#); GPU would sidestep the CPU spin-wait entirely (needs fast `fp64`).

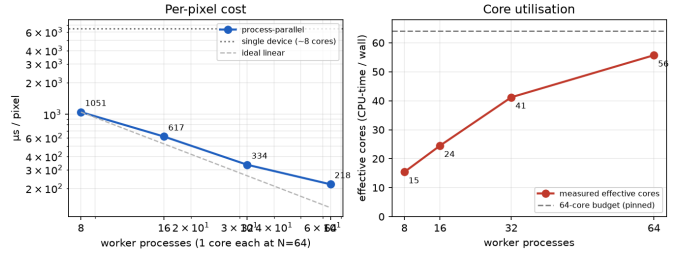


Figure 1: *Left:* per-pixel cost drops nearly linearly as the fixed 64-core budget is split into more, smaller workers, approaching the ideal; the single-device path (dotted) is  $\sim 30\times$  slower. *Right:* measured effective cores climb toward the 64-core budget, reaching 55.7 at  $64 \times 1$ .